

Article

SENS: Semantic-Aware Coalescing for High-Performance NVMe over TCP Storage Networks

Xinghan Qiao *, Lei Tian, Ge Hu and Xuchao Xie

College of Computer Science and Technology, National University of Defense Technology,
Changsha 410073, China

* Correspondence: qiaoxinghan@nudt.edu.cn

Abstract

In HPC systems and hyper-scale data centers, the adoption of high-performance NVMe SSDs and high-speed networks has shifted storage bottlenecks to the network stack. Under high-concurrency workloads, frequent interrupt processing exhausts CPU resources while protocol-level control–data dependencies in the NVMe over TCP write path introduce additional serialization penalties. Existing optimizations either require specialized hardware, dedicate CPU cores to user-space polling, or apply semantically blind batching that delays time-sensitive control messages. We present SENS, a Semantic-aware NVMe over TCP Scheduler embedded within the NVMe over TCP driver of the Linux kernel. SENS combines two mechanisms: (1) PDU vectorization, which aggregates discrete Protocol Data Units into memory vectors before network transmission, amortizing per-I/O system call overhead and reducing soft-interrupt frequency; and (2) instruction-aware dispatch, which detects control PDUs such as R2T and triggers an early flush of the aggregation window, mitigating the serialization penalty on the write path. A prototype evaluation with physical NVMe SSDs and 100 GbE networks shows that SENS saturates the SSD throughput ceiling using 4–5 CPU cores, halving the host-side core budget compared to the native TCP driver. With a RAMDisk backend that removes storage-media constraints, SENS sustains up to $2.5\times$ higher concurrent IOPS. These results show that exposing storage-protocol semantics to the batching layer improves the scalability of NVMe over TCP without additional hardware.

Keywords: NVMe over TCP; storage disaggregation; semantic-aware scheduling; network protocol

1. Introduction

Data centers are moving towards the disaggregated storage architecture, allowing compute and storage resources to scale independently and optimizing resource utilization [1–3]. In this environment, the NVMe over Fabrics (NVMe-oF) protocol has become a widely adopted industry standard for improving storage performance from end to end [4,5]. Of its transport options, NVMe over TCP offers a cost-effective, scalable way to develop large storage networks by making use of standard Ethernet infrastructure and the typical TCP/IP networking stack, removing the need for specialized hardware such as RDMA-capable Network Interface Cards (NIC).

The widespread adoption of 100 GbE networks and high-performance NVMe SSDs highlights an increasing performance gap between hardware bandwidth and host-side network stack processing [6]. Under highly concurrent, small-block storage I/O workloads,



Received: 11 March 2026

Revised: 20 April 2026

Accepted: 21 April 2026

Published: 23 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

high packet transmission rates cause standard NVMe over TCP implementations to generate frequent interrupt processing and kernel context switches. This interrupt overhead diverts CPU cycles from application workloads and limits overall storage performance.

Existing system-level optimizations generally fall into three categories: hardware offloading via Data Processing Units (DPU) or SmartNICs [7–10], user-space polling mechanisms like SPDK, and kernel-level network batching techniques such as i10 [11,12]. Hardware offloading achieves high throughput but introduces extra capital costs and vendor lock-in risks. User-space polling eliminates interrupt overhead but requires dedicated CPU cores for continuous busy-polling, wasting compute resources even under low I/O loads. Its kernel-bypass nature also breaks integration with VFS, page caches, and OS-level security isolation. Kernel-level network batching avoids these hardware and ecosystem limitations but primarily targets general network traffic, lacking awareness of underlying storage protocol semantics.

Specifically, kernel-level batching schemes aggregate all PDU types uniformly—control instructions and data payloads share the same accumulation window and timer. This reduces interrupt frequency but is semantically blind. In NVMe over TCP, the target multiplexes three outbound PDU types on each connection: C2HData (read payload), CapsuleResp (completion notification), and R2T (write-path control). The Out-Capsule write mode requires the host to block until it receives an R2T before transmitting data [13]. Uniform batching traps R2T behind bulk data in the aggregation window, creating a write-path serialization penalty that generic batching cannot address.

To solve this mismatch without additional hardware or dedicated polling cores, this paper proposes SENS, a semantic-aware scheduling layer that operates within the in-kernel NVMe over TCP driver. The main contributions are as follows:

PDU Vectorization: SENS aggregates multiple PDUs within the same connection into a vectored I/O structure prior to transmission. By submitting a batch of PDUs through a single vectored send operation, SENS amortizes the per-packet processing cost and reduces soft-interrupt frequency under high-concurrency workloads.

Instruction-Aware Dispatch: Through an asymmetric dual-threshold mechanism, SENS prioritizes critical control PDUs such as R2T, protecting them from batching-induced delays. This minimizes write pipeline stalls caused by R2T queuing and reduces write execution waiting times.

System-level Evaluation: We implemented SENS within the Linux kernel NVMe over TCP driver. On commodity servers with physical NVMe SSDs and a 100 GbE link, SENS saturates the target SSD with half the core count of native TCP. With a RAMDisk backend that removes media constraints, it reaches $2.5\times$ higher concurrent IOPS.

The rest of this paper is organized as follows. Section 2 introduces the background, protocol bottlenecks, and related work. Section 3 details the SENS architecture. Section 4 presents the implementation details. Section 5 evaluates the system performance. Section 6 discusses the applicability and future directions. Section 7 concludes the paper.

2. Background

This section covers the NVMe over TCP architecture (Section 2.1), its protocol-level bottlenecks (Section 2.2), related optimization approaches (Section 2.3), and the motivation for SENS (Section 2.4).

2.1. NVMe over Fabrics

NVMe-oF drives the evolution of storage disaggregation. Among its transport bindings, RDMA-based solutions dominate latency-critical domains such as AI training and high-performance computing due to their zero-copy and kernel-bypass capabilities. How-

ever, large-scale RDMA deployment in general-purpose data centers remains challenging. In practice, it is typically realized through RoCEv2, which requires a lossless fabric and careful configuration of Priority Flow Control (PFC) and Explicit Congestion Notification [14,15]. These requirements increase operational complexity and introduce risks such as congestion propagation and PFC deadlocks. In addition, specialized RDMA-capable NICs raise capital cost and integration overhead. These constraints hinder the adoption of RDMA in cost-sensitive, heterogeneous environments [16].

NVMe over TCP offers superior universality and cost efficiency. In a typical deployment, hosts and targets communicate through commodity TCP sockets, with per-core Submission and Completion Queues assigned to eliminate lock contention. End-to-end I/O operations are carried over standard Ethernet via PDUs, including command, data, and response messages. This allows scalable storage disaggregation without requiring specialized network hardware.

Despite these advantages, NVMe over TCP suffers from a widening mismatch between hardware throughput and host-side protocol processing capacity. The emergence of modern high-speed SSDs and 100 GbE+ networks has pushed raw I/O bandwidth to levels that exceed host-side processing capacity [17–20]. In contrast, single-core CPU performance and kernel-level TCP/IP processing efficiency have improved relatively slowly. Under high-concurrency workloads, the overheads of packet processing and context switching dominate execution time, causing the kernel network stack to become the throughput bottleneck.

2.2. Protocol Bottlenecks

The standard NVMe over TCP implementation exposes two structural bottlenecks under high concurrency workloads: interrupt processing overhead from per-PDU packet handling, and an R2T control-flow dependency on the write path.

Interrupt Overhead: In the standard Linux implementation, a single NVMe over TCP request requires exchanging multiple PDUs. For a read operation, the host issues a Command PDU; the target then returns a C2HData PDU with the payload, followed by a Response PDU. Under small-block (e.g., 4 KB) random workloads, this frequent message exchange generates extremely high packet rates. Each incoming packet triggers a NIC hardware interrupt, waking kernel softirq handlers to process the TCP/IP stack [21–26]. As I/O concurrency scales, this packet processing forces the CPU into frequent context switches across user space, the block layer, and network drivers. Eventually, interrupt handling and protocol parsing consume the majority of CPU cycles. Recent profiling quantifies this systemic overhead. Kang et al. [6] show that TCP stack processing and queue pair interactions generate severe CPU load in multi-tenant NVMe over TCP environments. Earlier characterizations by Guz et al. [4] and Jia et al. [10] similarly confirm that high-speed TCP storage traffic heavily taxes softirq execution. Under sustained load, application workloads are deprived of compute resources, forcing incoming packets to queue in kernel sockets and inflating end-to-end latency.

R2T Dependency: Beyond network stack overhead, the NVMe over TCP protocol introduces strict structural constraints on data writes. The specification defines two transmission modes: In-Capsule and Out-Capsule. Hosts embed small payloads directly within the initial Command PDU. Large writes, however, mandate the Out-Capsule mode. In Out-Capsule mode, the host cannot transmit data immediately after issuing a write command. It must wait for the target to allocate memory buffers and return an R2T PDU. Only after parsing the offset and length specified in the R2T can the host dispatch the actual H2CData PDUs. This mechanism strictly couples data transmission to the arrival and processing of control messages [27].

In production environments, these two bottlenecks compound each other. When softirq processing overwhelms the host CPU, the kernel network stack handles incoming R2T messages at fluctuating speeds. Because the host strictly blocks on the R2T to proceed, any microsecond delay in R2T processing adds directly to the write request's lifecycle. Delayed R2T processing directly extends the write request lifecycle, reducing the effective concurrency of the write path.

2.3. Related Work

To mitigate protocol overhead and scheduling bottlenecks in high-speed NVMe over TCP systems, recent system-level optimizations generally fall into three categories: hardware offloading, user-space bypass, and kernel-level batching.

Hardware Offloading: Delegating the NVMe-oF data path to DPUs or SmartNICs has gained traction in recent system designs. The Scalio architecture offloads JBOF key-value processing to DPUs [7], and Xu et al. characterize SmartNIC-based NVMe-oF target offloading under production-like conditions [8]. These studies show that moving TCP/IP processing and NVMe encapsulation off the host CPU reduces protocol-processing overhead and raises throughput. The trade-off is deployment cost: DPU procurement is expensive and vendor-specific firmware creates lock-in risks, limiting adoption in cost-sensitive, large-scale data center environments [28–31]. Similarly, Cheng et al. [32] design a memory-efficient programmable parser for multi-tenant terabit networks, but this approach also relies on FPGA hardware.

User-space Bypass: Frameworks such as SPDK move the entire storage and network stack into user space, replacing interrupts with busy-polling to eliminate kernel context switches. While this achieves peak throughput, it dedicates CPU cores exclusively to polling regardless of I/O load and bypasses the kernel, breaking integration with VFS, page caches, and OS-level security isolation [33–35]. These constraints make it poorly suited for multi-tenant environments that require dynamic resource sharing.

Kernel-level Batching: Within the kernel, Linux NAPI and adaptive interrupt coalescing dynamically adjust driver-level interrupt rates to lower CPU load [36–39]. The i10 architecture [11] reduces overhead by batching NVMe-oF PDUs and delaying doorbell ringing, reaching throughput-per-core comparable to RDMA on commodity hardware. However, these kernel-level approaches apply uniform batching to all PDU types—control instructions and data payloads share the same aggregation window and timer [40]. This uniform treatment cannot protect time-sensitive control messages such as R2T from queuing behind bulk data segments.

2.4. Motivation

The approaches reviewed above each address part of the problem but leave a gap. Hardware offloading reduces CPU load but adds capital cost and vendor dependency. User-space polling removes interrupt overhead but dedicates cores to busy-waiting regardless of I/O load. Its kernel-bypass design also breaks integration with VFS, page caches, and OS-level security isolation. Kernel batching avoids both issues but treats all PDU types uniformly, leaving the R2T serialization penalty unresolved.

Despite their differences, these approaches share a common limitation: they do not expose NVMe-oF PDU semantics to the critical-path scheduling. In NVMe over TCP, the target sends data payloads (C2HData), completion notifications (CapsuleResp), and write-gating controls (R2T) onto the same target-to-host connection. Among these, only R2T directly gates Out-Capsule writes: the host cannot transmit data until it receives the corresponding R2T. A batching mechanism that cannot distinguish R2T from other

PDU may trap it behind bulk traffic, increasing write-path stalls. Resolving this mismatch requires making NVMe-oF PDU semantics visible to the scheduling layer.

SENS incorporates this semantic awareness into the kernel NVMe over TCP driver. It applies PDU vectorization to reduce interrupt frequency on both host and target sides, and adds an instruction-aware dispatch mechanism at the target side that distinguishes control PDUs from data PDUs in the outbound stream. R2T PDUs are tracked by an independent counter and timer so that they flush ahead of bulk data, reducing write-path stalls without disabling batching for other traffic.

3. SENS Design

SENS is a scheduling layer within the kernel NVMe over TCP driver that interposes on the PDU path between the block layer and the TCP stack. This section describes the overall architecture (Section 3.1), PDU Vectorization (Section 3.2), Instruction-Aware Dispatch (Section 3.3), and the analytical model (Section 3.4).

3.1. Architecture Overview

SENS is implemented as a scheduling layer within the kernel NVMe over TCP driver. As illustrated in Figure 1, it is positioned between the block layer and the TCP/IP network stack on both the host and target nodes. This placement enables SENS to intercept and schedule NVMe PDUs in both directions while remaining compatible with the existing storage stack.

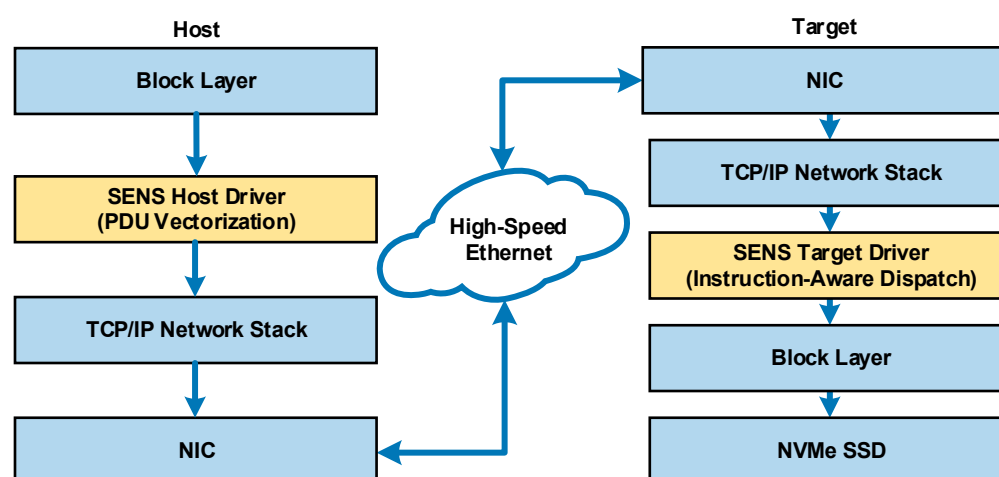


Figure 1. End-to-end system architecture and module integration of SENS.

Within this architecture, the two core mechanisms work together along the data path. During outbound transmission, SENS intercepts discrete PDUs issued by the block layer and applies PDU Vectorization to consolidate them into unified memory vectors before passing them to the TCP/IP stack. On the target side, SENS enforces an Instruction-Aware Dispatch policy. SENS inspects the semantic type of each intercepted PDU to distinguish time-sensitive control instructions from regular data, so that critical control flows can bypass accumulation delays introduced by bulk batching. These two mechanisms form the SENS scheduling pipeline.

3.2. PDU Vectorization

To eliminate the per-PDU submission overhead analyzed in Section 2.2, SENS introduces a bidirectional PDU Vectorization mechanism within the kernel data path that coalesces discrete PDUs into unified PDU Groups (PG), as illustrated in Figure 2.

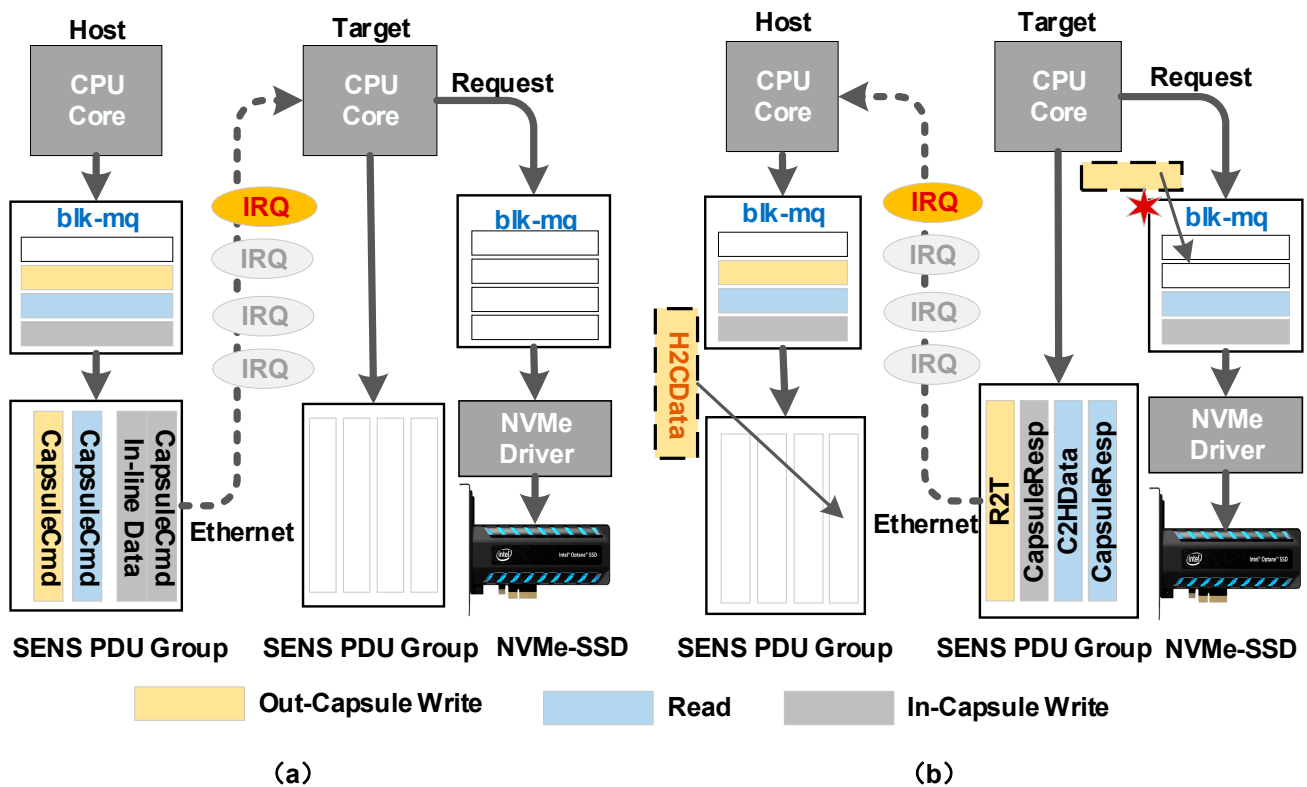


Figure 2. Data flow and PDU vectorization across bidirectional endpoints. (a) Host-side PDU vectorization; (b) Target-side PDU vectorization.

Instead of relying on intermediate buffer staging, SENS uses a vectored I/O structure (*kvec*) to record the memory addresses and lengths of discrete PDUs. A PG capacity counter tracks the number of aggregated PDUs within the current batch. PDUs are appended to the array until the accumulated capacity hits a predefined threshold or a fallback timeout expires. SENS then dispatches the entire vector via a single *kernel_sendmsg* invocation. This batching strategy converts dozens of fine-grained submissions into a single protocol-stack transaction, reducing interrupt frequency and per-packet processing overhead.

This basic grouping mechanism operates bidirectionally based on distinct storage semantics. At the host node (Figure 2a), SENS aggregates outbound requests—primarily *CapsuleCmd* and *H2CData* PDUs—into a unified batch. Once the batch is full, SENS transmits the entire vector in a single *kernel_sendmsg* call, significantly reducing the number of interrupts at the target and allowing the target CPU to process multiple PDUs consecutively.

Upon receiving the batch, the target engine parses the PDUs and processes them according to their command types. For read requests and In-Capsule writes, the target initializes local block I/O operations and constructs *Response* and *C2HData* PDUs. Conversely, for Out-Capsule write requests, the target constructs an *R2T* PDU to notify the host to begin data transmission. To reduce reverse-path interrupt frequency, the target side employs the same vectorized mechanism to coalesce these outgoing *C2HData*, *CapsuleResp*, and *R2T* PDUs into a unified batch before returning them to the host (Figure 2b).

While bidirectional batching improves throughput, uniform aggregation at the target side delays control PDUs such as *R2T*. Because Out-Capsule writes strictly depend on the timely arrival of *R2T* messages, trapping these *R2T* PDUs within a standard aggregation window directly stalls the write pipeline and inflates end-to-end write latency. This problem is specific to the target side: the host's outbound traffic consists predominantly of *CapsuleCmd* and *H2CData* PDUs with no mixed control-data contention, whereas the target multiplexes *C2HData*, *CapsuleResp*, and *R2T* PDUs onto the same connec-

tion. The resulting semantic conflict motivates the instruction-aware dispatch mechanism described next.

3.3. Instruction-Aware Dispatch

Since R2T PDUs strictly gate the transmission of large write transfers, queuing these control messages within a standard aggregation window exacerbates the write-path serialization penalty and inflates end-to-end latency.

To reduce R2T queuing delay, SENS adds an instruction-aware dispatch mechanism specifically at the target side (as illustrated in Figure 3). The scheduler maintains an asymmetric dual-threshold and dual-timer mechanism: a global aggregation capacity with its default timer for standard payloads, and an independent control capacity with a dedicated microsecond-scale timer for time-sensitive instructions.

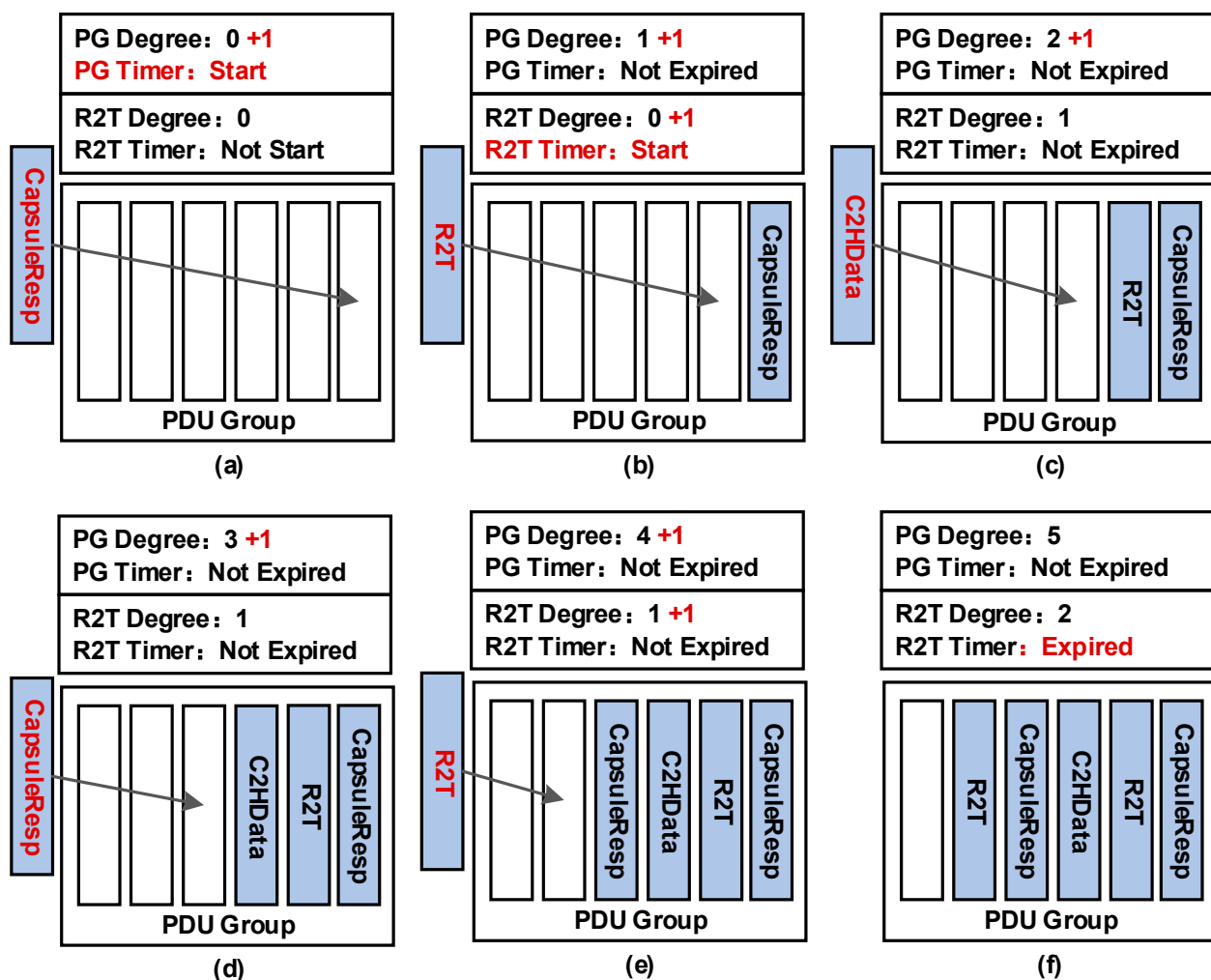


Figure 3. PDU Aggregation and PG Send Mechanisms of Target. (a) Initial aggregation state without R2T; (b) First R2T arrival activates the dedicated R2T timer; (c) Subsequent PDUs continue to be aggregated with the R2T pending; (d) Mixed PDUs further accumulate in the PG; (e) Another R2T arrives and updates the control-state tracking; (f) The dedicated R2T timer triggers an early flush of the PG.

During transmission, the target classifies outgoing PDUs by type. When standard data and response PDUs enter the group, they increment only the global PG capacity and trigger the standard PG timer. Conversely, upon detecting critical control instructions such as R2T PDUs, SENS updates the global capacity and simultaneously accumulates these

instructions into the independent control capacity and arms the dedicated control timer. As subsequent PDUs are appended, the engine continuously evaluates all tracking parameters.

The state machine preempts the standard aggregation cycle and immediately flushes the aggregated vector whenever any triggering condition is met: either the global PG capacity or the independent control capacity reaches its respective threshold, or either of the two timers expires. Configured with a significantly tighter threshold and a shorter timeout than the global limits, this independent tracker reduces the queuing time for control PDUs. The control timer operates independently—its initialization does not affect the global PG timer.

This asymmetric design reduces the queuing delay for control messages. Faster R2T delivery reduces the write-path serialization penalty while the data-class PDUs still benefit from batching.

3.4. Analytical Model

The two mechanisms described above can be analyzed using a queuing-theoretic framework. We model the interrupt reduction from PDU Vectorization first, then examine how Instruction-Aware Dispatch lowers R2T queuing delay on the write path.

3.4.1. Modeling Interrupt Reduction

Consider a single NVMe over TCP connection at the target side where outbound PDUs arrive at an aggregate rate λ (PDUs per second). We model this arrival process as Poisson, which is reasonable under high-concurrency random I/O where many independent requests overlap in time.

The PG operates as a timer-gated bulk server. PDUs accumulate until one of two conditions triggers a flush: either B PDUs have arrived (capacity-triggered) or the doorbell timeout T_d has elapsed since the first PDU entered an empty PG (timer-triggered). Each flush produces one `kernel_sendmsg` call and, at the receiving side, one NIC interrupt.

In native NVMe over TCP, every PDU is sent individually, so the interrupt frequency equals λ . With PDU Vectorization, multiple PDUs share a single transmission, and the interrupt frequency drops accordingly. To quantify this reduction, let T_B be the time needed for $B - 1$ additional PDUs to arrive after the first one. Since inter-arrival times are exponential with rate λ , T_B follows an *Erlang*($B - 1, \lambda$) distribution. A flush happens at time $\tau = \min(T_B, T_d)$ after the first arrival, giving an effective batch size of:

$$E[B_{eff}] = B \cdot P(T_B \leq T_d) + E[1 + N(T_d) | T_B > T_d] \cdot P(T_B > T_d), \quad (1)$$

where $N(T_d)$ is the number of Poisson arrivals in the interval T_d .

Two regimes emerge from Equation (1). Under high concurrency, λ is large enough that $T_B \ll T_d$ with high probability, meaning the PG fills up before the timer triggers. In this regime, $E[B_{eff}]$ approaches B and the interrupt frequency drops from λ to roughly λ/B —a B -fold reduction. With the default PG capacity $B = 16$, this corresponds to a $16\times$ reduction in interrupt frequency.

Under low concurrency, the PG rarely fills to capacity and most flushes are timer-triggered. The effective batch size becomes approximately $1 + \lambda \cdot T_d$, which approaches 1 when λ is very small. Batching provides little benefit in this regime. This matches the observation from i10 [11] that batching adds latency at low loads without improving throughput.

We can also derive the per-PDU queuing delay inside the PG. In the capacity-triggered regime, the k -th PDU entering a batch waits for the remaining $B - k$ PDUs to arrive before the flush. Averaging over all positions in the batch:

$$E[W_{batch}] = \frac{B-1}{2\lambda}. \quad (2)$$

This delay shrinks as λ grows—at high arrival rates, the PG fills quickly and each PDU spends little time waiting. Equation (2) captures the core tradeoff of batching: larger B reduces interrupts but increases per-PDU residence time in the PG.

3.4.2. R2T Queuing Delay on the Write Path

For Out-Capsule writes, the host blocks until an R2T arrives from the target. We decompose the end-to-end write latency as shown in Equation (3):

$$T_{write} = T_{cmd} + T_{proc} + W_{r2t} + T_{net} + T_{data}. \quad (3)$$

Here T_{cmd} is the command transmission time, T_{proc} is the target processing time, W_{r2t} is the R2T queuing time inside the target-side PG, T_{net} is the network transit time, and T_{data} covers the H2CData transfer. Among these terms, the target-side asymmetric dispatch only affects W_{r2t} . Because the host strictly blocks on the R2T, any increase in W_{r2t} extends the write request lifetime directly.

We partition the target-side outbound PDUs into two classes: control-class (R2T) and data-class (C2HData and CapsuleResp). Let p denote the fraction of control PDUs in the total stream. Under uniform batching (PGIC), all PDUs share a single threshold B , and an R2T waits on average for $(B-1)/2$ PDUs to complete before the flush.

SENS adds an independent control counter c_2 . The PG flushes after whichever threshold is reached first—either B total PDUs or the expected number of total PDUs needed to accumulate c_2 control PDUs, which equals c_2 divided by p . The R2T delay reduction ratio between SENS and PGIC is:

$$R = \frac{\min(B, c_2/p) - 1}{B - 1}. \quad (4)$$

Three cases arise from Equation (4). When p exceeds c_2 divided by B , the control threshold triggers first. This is the common case for writes. A pure Out-Capsule write workload produces roughly one R2T and one CapsuleResp per command, giving $p \approx 0.5$. With $c_2 = 4$ and $B = 16$, the control threshold triggers at 8 PDUs, yielding $R = 7/15 \approx 0.47$ —a 53% reduction in R2T queuing delay. When p is below this boundary (read-heavy workloads), the global threshold triggers first and R equals 1; SENS behaves identically to PGIC. When no R2T PDUs exist (pure reads), W_{r2t} is absent and the Equation (4) does not apply.

Since the expected base queuing time under PGIC is $(B-1)/(2 \cdot \lambda_{total})$, the per-request write latency reduction follows from Equations (3) and (4) as:

$$\Delta T_{write} = (1 - R) \cdot \frac{B-1}{2 \cdot \lambda_{total}}. \quad (5)$$

With $R \approx 0.47$, approximately half of the R2T queuing component is removed from each write request. The realized throughput gain depends on how much of the total write latency is attributable to W_{r2t} —a fraction that grows as the storage backend becomes faster and network overhead shrinks.

These predictions are consistent with the experimental results. As shown in Figure 4, the IOPS gap between SENS and PGIC is largest for 16 KB random writes—up to 30% at moderate concurrency (5–10 cores)—where R2T gating is the primary bottleneck. For 4 KB random reads, the two schemes show comparable throughput, as expected since no R2T PDUs are involved in a pure read workload.

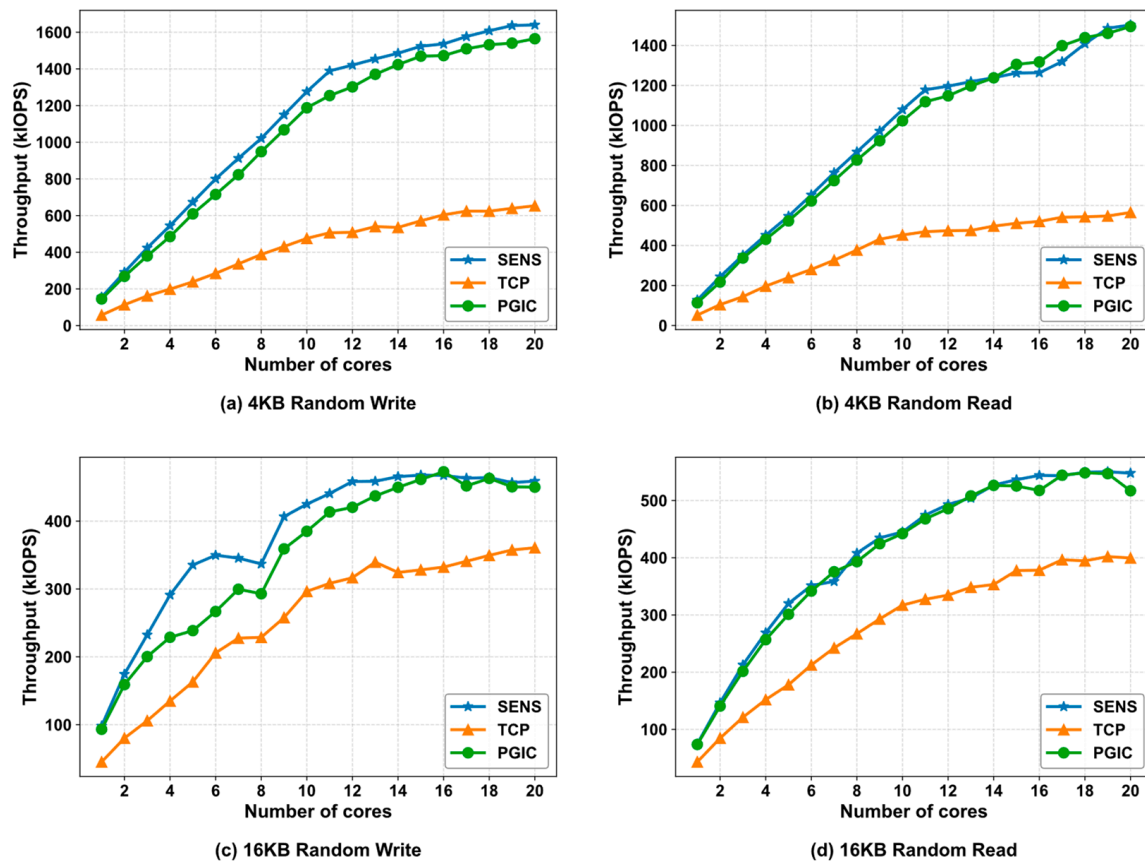


Figure 4. Read and Write IOPS Comparisons with RAMDisk.

4. Implementation

We implemented the SENS prototype within the in-kernel NVMe over TCP driver of the Linux kernel 4.20. We selected this kernel version to align with the evaluation environment of i10 [11], enabling comparison under the same driver codebase and aggregation parameters. The two protocol-level bottlenecks that SENS targets—per-PDU submission overhead and the R2T gating dependency—are defined by the NVMe-oF specification [27] and remain structurally present in later kernel versions. Within this driver, SENS adds a lightweight scheduling layer between the generic block layer and the TCP/IP stack. The modification interposes on both the data and control paths, adding approximately 460 lines of code: about 210 on the host side and 250 on the target side.

Within the kernel queue context, we introduced a kvec-based PDU Group (PG) array buffer on each connection path for vectorized transmission. On the host side, SENS intercepts the native I/O wake-up path by registering hrtimer instances as lightweight delayed doorbells. These timers serve as deferred aggregation windows rather than a new execution model: incoming packets are still processed through standard interrupt handlers, and hrtimer callbacks simply wake the queue transmission path when the aggregation timeout expires. Transmission is then governed by PG capacity thresholds and timer expiration rather than immediate per-PDU submissions, while the overall interrupt-driven scheduling model remains intact. All extended data structures are allocated during the native alloc_queue phase and safely reclaimed through explicit pointer traversal in free_queue.

On the target side, resolving control-flow latency requires prioritized handling of R2T and Response paths. We introduced an independent control-plane counter alongside the global PG capacity. The counter tracks the number of control PDUs accumulated in the current batch, and is compared against a dedicated threshold c_2 . When the target enqueues a control PDU, both the global PG capacity and the control-plane counter are incremented.

For ordinary data batching, only the global PG capacity is updated. The transmission loop evaluates these two capacities asymmetrically. When the control-plane counter reaches $c2$, the state machine sets `send_now = true`, triggering an immediate flush regardless of the remaining global PG capacity. This prevents control messages from being delayed behind bulk data aggregation under high concurrency.

The base aggregation parameters follow `i10` [11]: a PG capacity of $B = 16$ PDUs and a doorbell timeout of $T_d = 50 \mu\text{s}$. SENS introduces one additional parameter: the control-capacity threshold $c2$ with an associated timer T_c . Equation (4) in Section 3.4 shows that $c2$ only takes effect when $c2$ is divided by p is less than B . For a write-intensive workload with $p \approx 0.5$, this means any $c2$ at or above 8 makes the control threshold redundant. At $c2 = 4$, the PG flushes at 8 PDUs, retaining half of the full batching benefit while cutting R2T queuing delay by 53%. Reducing $c2$ further to 2 or 1 would halve or eliminate the batching benefit in exchange for only marginal additional delay reduction. We therefore select $c2 = 4$. The associated control timer T_c is set to $10 \mu\text{s}$, one-fifth of T_d , and operates independently of the global PG timer.

SENS includes two safeguards for transmission robustness and memory safety. First, because `kernel_sendmsg` may return a short send when the TCP send buffer is full, the dispatch routine maintains an `iov_offset` cursor. If a send operation is incomplete, the exact offset within the vector array is recorded to guarantee correct resumption during the next wake-up event, preventing byte-stream inconsistency. Second, on the target side, a deferred-release barrier delays command reclamation and page unmapping until the entire PG batch has been transmitted. Without this barrier, the target may reclaim a command structure or unmap its data pages while the corresponding PDU is still queued in the `kvec` awaiting transmission, leading to use-after-free conditions.

5. Evaluation

This section evaluates SENS across four dimensions: throughput scalability (Section 5.2), end-to-end latency (Section 5.3), CPU efficiency (Section 5.4), and mixed read/write workload performance (Section 5.5).

5.1. Experimental Setup

The evaluation cluster consists of two servers configured as the host and target. The nodes are directly connected via a 100 GbE link using Mellanox ConnectX-5 adapters configured in Ethernet mode over an EDR optical fiber. The target node is equipped with a 1.6 TB U.2 NVMe SSD. To isolate protocol stack overhead from storage-media limitations, we additionally deploy a high-performance RAMDisk as the backend device. This configuration allows us to examine network and kernel processing behavior independent of flash latency constraints.

The detailed hardware and software configuration of the testbed is summarized in Table 1. Each server is equipped with dual Intel Xeon E5-2660 v3 processors (2.60 GHz) and 128 GB of memory. Pairing these CPUs with a 100 GbE network reflects a configuration in which network and storage bandwidth significantly exceed single-core processing growth. Under this setup, interrupt handling and context-switch overhead in the native network stack become more visible, providing a controlled environment to evaluate the effects of SENS. The software environment runs CentOS 7.7.1908 with Linux kernel 4.20 and the `OFED-5.0-2.1.8` driver.

We compare SENS with three baselines:

NVMe-Local: Direct local access to the NVMe SSD, bypassing the network stack. This configuration serves as a reference for device-level performance.

NVMe-TCP: The unmodified Linux NVMe over TCP implementation, representing the standard kernel baseline.

PGIC: SENS with instruction-aware dispatch disabled. All PDU types share a single aggregation threshold (PG capacity 16, timeout 50 μ s) and timer, reproducing the uniform batching policy of i10 [11]. The gap between PGIC and SENS isolates the contribution of instruction-aware dispatch.

Workloads are generated using fio (v3.7) with 4 KB and 16 KB random read and write patterns. The devices are managed via nvme-cli (v1.8.1). To evaluate scalability and CPU consumption, we incrementally scale the concurrent I/O workload from 1 to 20 CPU cores.

Table 1. Configuration for SENS Performance Evaluation.

Hardware Platform	Model/Specification	Software Environment	Version
Processor	Dual Xeon E5-2660 v3	Linux Kernel	4.20
Memory	128 GB DDR4	NIC Driver	OFED 5.0-2.1.8
NIC	Mellanox CX-5 (100 GbE)	nvme-cli	1.8.1
NVMe SSD	1.6 TB PCIE 4.0 U.2	Benchmark Tool	fio-3.7

5.2. Throughput Scalability

Figure 4 presents throughput scalability under different I/O granularities using the RAMDisk backend. By removing the performance ceiling imposed by physical flash media, these experiments isolate the processing capacity and architectural constraints of the software network stack.

Under highly concurrent small I/O workloads, the native TCP implementation exhibits clear context-switching and interrupt handling overhead. Even when scaling to 20 CPU cores, throughput stalls at approximately 653 kIOPS. In comparison, both SENS and the PGIC baseline scale more consistently with increasing core counts, because PDU vectorization consolidates multiple PDUs into single send operations, reducing the number of system calls and associated interrupts. At 20 cores, SENS reaches 1639 kIOPS, a $2.5\times$ improvement over the native NVMe over TCP implementation.

For larger payloads, protocol-level flow-control constraints introduce stronger transmission dependencies. Because PGIC does not differentiate control PDUs from data segments, R2T messages queue behind coalesced data, stalling the host write pipeline. The effect is visible in the 16 KB write curve: between 5 and 10 cores PGIC ranges from roughly 239 to 385 kIOPS, while SENS ranges from 335 to 425 kIOPS—an advantage of up to 30% at moderate concurrency. Both schemes eventually converge near 460 kIOPS, but SENS reaches this saturation region 3 cores earlier, reducing the CPU investment required to saturate the write path.

These results indicate that incorporating storage-level semantics into the batching layer improves scalability when protocol dependencies become dominant.

5.3. Latency Analysis

PDU batching introduces additional queuing delay by design, as it aggregates requests before transmission. Figure 5 reports the 4 KB average latency under varying concurrency levels and shows how different designs bound this overhead.

In the RAMDisk configurations (Figure 5a,c), native NVMe over TCP latency grows sharply with concurrency due to severe interrupt and context-switching overheads. Both SENS and PGIC flatten this curve via batching, yet their dispatch mechanics yield different latency bounds. Relying on static timers and global thresholds, PGIC may delay control PDUs when the global timer has not yet expired. SENS avoids this by flushing the PG

when the control counter reaches c_2 , independent of the global timer, lowering average latency under high concurrency.

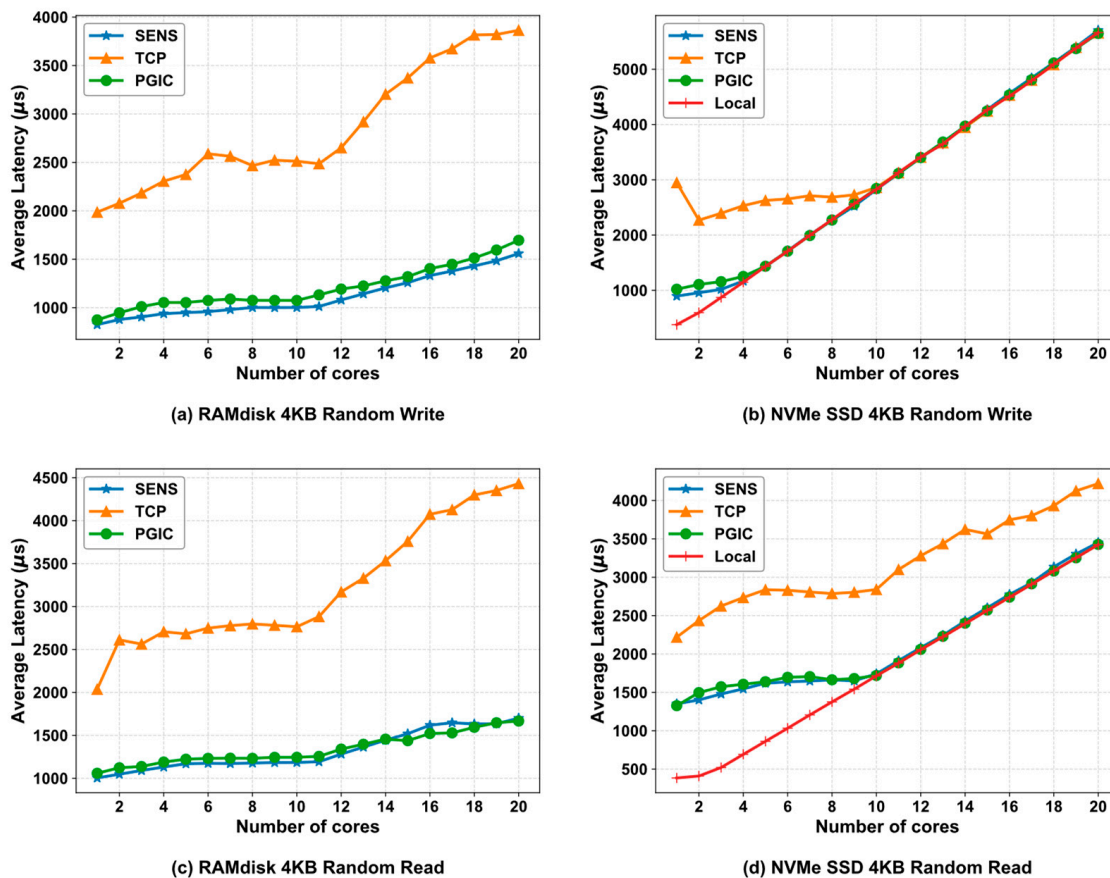


Figure 5. End-to-end Average Latency Analysis.

The NVMe SSD evaluations (Figure 5b,d) expose system behavior under strict physical media constraints. Beyond specific concurrency thresholds (approx. 5 cores for writes and 10 for reads), SENS and PGIC converge tightly with the Local NVMe baseline, exhibiting a near-linear latency increase dictated by Little's Law ($L = \lambda \times W$). Because SENS saturates the SSD's throughput limit ($\lambda \approx 450$ kIOPS) early, allocating additional CPU threads strictly increases the number of in-flight requests (L). Consequently, the average wait time (W) scales proportionally, confirming that the performance bottleneck has shifted entirely from the protocol stack to the physical storage device.

Under low-to-medium concurrency (1–8 cores in Figure 5b), native TCP shows a different pattern. With fewer cores, interrupt handling and softirq processing consume most of the available CPU time, causing both outgoing commands and incoming responses to queue in the kernel network stack. Software overhead—not device service time—dominates the response latency. As concurrency increases beyond 10 cores, the additional CPU capacity absorbs the softirq load, and native TCP latency converges with the device-limited curve.

5.4. CPU Efficiency

In multi-tenant data centers, CPU efficiency is a key metric because it determines how many cores remain available for tenant workloads. Figure 6 illustrates the performance bounds using the physical NVMe SSD, providing a direct metric for resource efficiency.

According to the local I/O baseline, the target SSD imposes a physical limit of approximately 450 kIOPS for 4 KB random writes (Figure 6a). The native TCP stack requires up to 11 CPU cores to approach this ceiling. SENS saturates the same device using only

4–5 cores. A similar trend holds for 4 KB random reads (Figure 6b): SENS reaches the SSD's read throughput limit with significantly fewer cores than native TCP. For 16 KB workloads (Figure 6c,d), all schemes converge more quickly because larger transfer sizes naturally amortize per-I/O overhead, narrowing the efficiency gap. In practical terms, SENS halves the CPU core budget needed to saturate this SSD. On the tested hardware, this frees 5–6 cores from protocol processing, improving overall host utilization at no extra hardware cost.

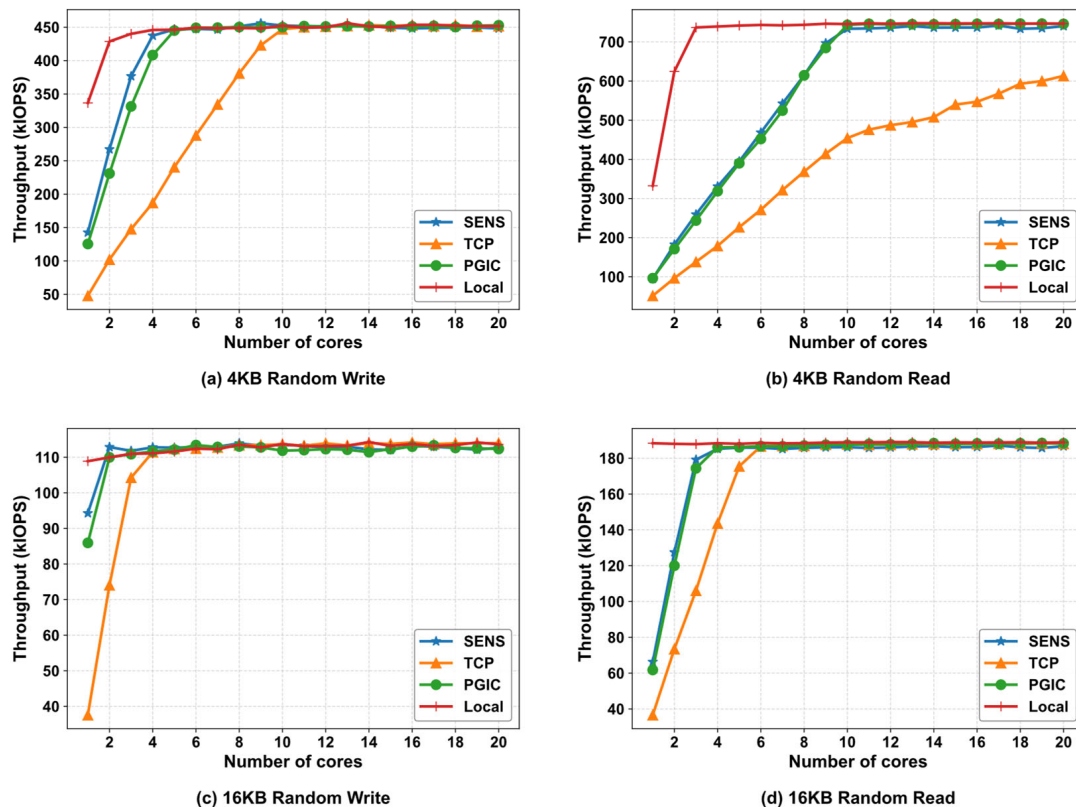


Figure 6. Read and Write IOPS Comparisons with SSD.

To examine the sources of these CPU savings, we conducted a CPU time breakdown under 16 KB random writes. CPU time is divided into User, System, and SoftIRQ components, and context switches per million I/Os (M/MIO) are recorded (Figure 7). The native TCP stack exhibits high overhead, consuming 30.7 K ticks/MIO on the host and 7.5 K ticks/MIO on the target, mainly in System and SoftIRQ processing. Both PGIC and SENS reduce CPU usage—host-side CPU time drops by approximately 70% and target-side by around 61% relative to TCP.

SENS incurs slightly higher CPU time than PGIC because the control threshold c_2 triggers flushes at an effective batch size of 8, increasing the number of flush operations per I/O compared to PGIC's full batch of 16. Its primary benefit is in scheduling efficiency: context switch frequency is reduced by 24.4% on the host and 15.2% on the target compared to PGIC. Under this workload, PGIC's uniform batching delays R2T messages, forcing host threads to block frequently. By preemptively flushing R2T PDUs, SENS mitigates write-path stalls, reducing the frequency of thread sleep/wakeup cycles and improving end-to-end I/O execution. This is consistent with Equation (5), which predicts that instruction-aware dispatch removes approximately half of the R2T queuing component from each write request, reducing pipeline stalls that would otherwise trigger additional context switches.

These results indicate that the CPU efficiency gains of SENS are mainly due to reduced context switching from fewer write-pipeline stalls, rather than lower per-I/O CPU time.

PDU vectorization reduces interrupt overhead, while instruction-aware dispatch improves handling of time-sensitive write-path control messages.

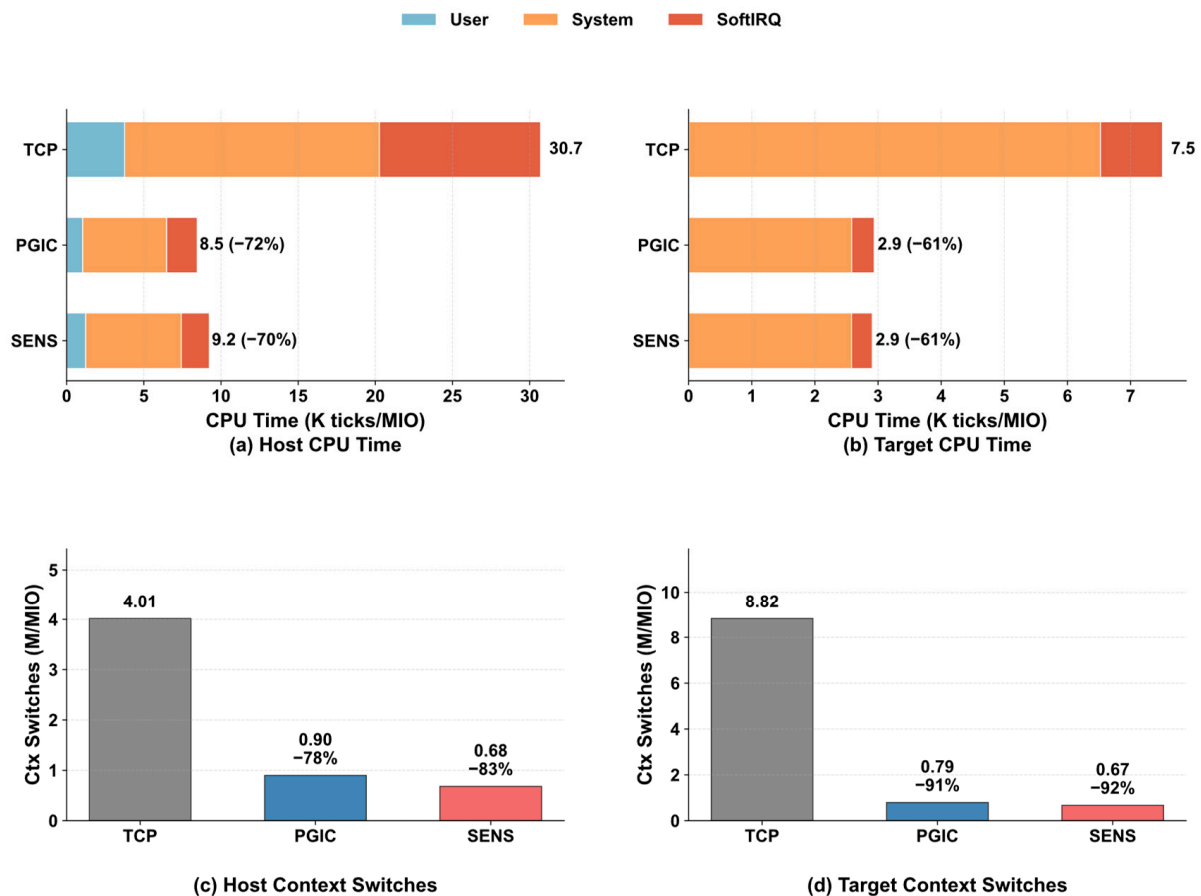


Figure 7. Normalized CPU cost breakdown for 16 KB random writes.

5.5. Mixed Read/Write Workloads

We evaluated SENS under mixed random read/write workloads to examine its robustness and performance relative to the native NVMe over TCP implementation. Figure 8 reports system throughput with varying concurrency, for request sizes of 4 KB and 16 KB and read ratios of 90% and 50%. SENS consistently outperforms the native TCP baseline across all configurations. For 4 KB workloads at a 50% read ratio, SENS reaches a maximum relative improvement over native TCP of 83.1%. For 16 KB workloads under the same read ratio, SENS achieves a maximum relative improvement of 40.7%. These results indicate that PDU vectorization effectively reduces interrupt overhead and improves throughput under high-concurrency mixed workloads.

To isolate the effect of instruction-aware dispatch, we compared SENS with PGIC. For 4 KB workloads, the two schemes perform similarly. This reflects the protocol behavior: small requests primarily use In-Capsule transfers, which do not trigger R2T messages, leaving instruction-aware dispatch inactive. SENS therefore falls back to standard batching without adding scheduling overhead.

For 16 KB workloads requiring Out-Capsule transfers, SENS shows increasing advantage over PGIC as the proportion of write operations grows. At 90% read, the maximum throughput gain is 3.7%, while at 50% read, it reaches 7.87%. The smaller gap compared to the pure-write results in Section 5.2 is expected: at 50% read ratio, only half of the I/O operations involve R2T exchanges, reducing the fraction of traffic that benefits from instruction-aware dispatch. This trend is consistent with Equation (4): as the write fraction

increases, the control PDU fraction p rises, making the control threshold c_2 more effective at reducing R2T queuing delay.

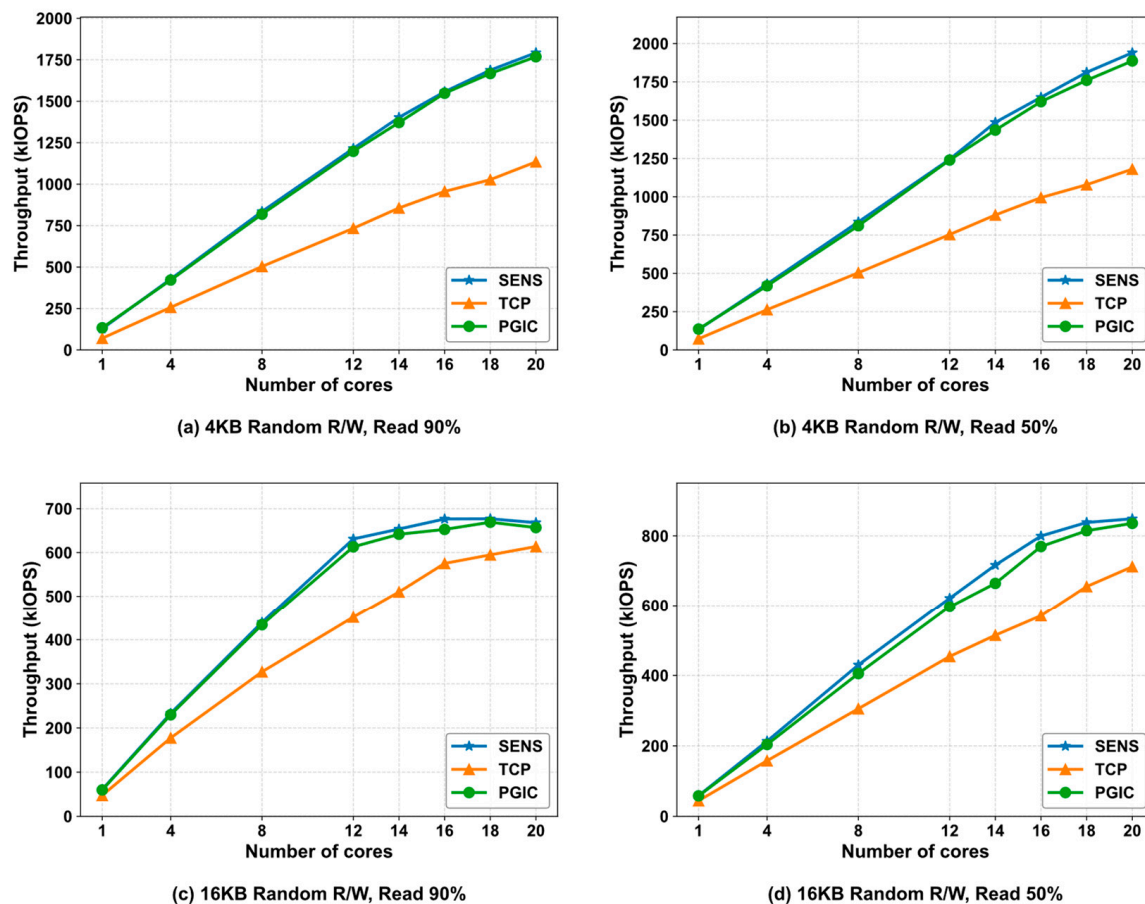


Figure 8. Throughput under mixed Read/Write workloads.

6. Discussion

Applicability and Limitations: The gains reported in Section 5 are most visible under highly concurrent, small-block random I/O. As request size grows, per-I/O processing overhead shrinks relative to data transfer time. Three scenarios limit the effectiveness of SENS. First, large sequential workloads generate low PDU rates; the native TCP stack already saturates the link with minimal concurrency, and Equation (2) in Section 3.4 predicts that the queuing delay grows under low λ while the interrupt reduction diminishes. Second, low-concurrency workloads have a similar effect—the PG is predominantly timer-triggered, so batching adds latency without meaningful throughput gain. Third, read-heavy workloads contain few or no R2T PDUs on the critical path. In this case, the control threshold provides no benefit, and SENS degrades gracefully to PGIC behavior, as predicted by Equation (4).

Under mixed sequential and random I/O, the sequential component lowers the aggregate PDU rate because each large transfer generates fewer PDUs per byte. The random component continues to benefit from batching and R2T prioritization. The net gain falls between the two extremes.

The asymmetric dispatch does not introduce fairness issues across connections. Each connection maintains its own PG with independent thresholds and timers. An early control flush reduces the effective batch size on that connection but does not affect other connections. For tenants sharing a single connection through connection pooling, the

dispatch applies uniformly to the mixed traffic. Per-tenant differentiation within a shared connection would require per-flow R2T tracking, which we leave for future work.

Hardware Generalizability: The current evaluation uses Xeon E5-2660 v3 processors and Linux kernel 4.20. On newer processors, the absolute interrupt cost per PDU may be lower, but network speeds have also increased to 200–400 GbE. Kang et al. [6] profile NVMe over TCP on a newer platform and report that TCP stack processing and softirq handling still consume a large fraction of host CPU time. This suggests that the ratio of per-PDU overhead to available CPU capacity has not changed fundamentally. The two bottlenecks that SENS targets—per-PDU interrupt overhead and R2T serialization—are defined by the NVMe-oF specification [27] and remain structurally present in later kernel versions. Validating SENS on newer kernel versions and hardware platforms is a direction for future work.

Towards Hardware-Software Co-Design: As more data centers adopt SmartNICs and DPUs for network stack offloading, the software-native design of SENS remains compatible with this trend. The proposed scheduling mechanism introduces minimal computational complexity. The core scheduling logic—a micro-state machine differentiating control flows from data flows—is compact enough to be implemented in SmartNIC firmware or storage controller pipelines in future work. Such co-design could further reduce host CPU involvement while preserving control-flow prioritization within the NIC, mitigating head-of-line blocking on the write path. Programmable data plane architectures point to a similar direction: Cheng et al. [32] achieve terabit-scale packet parsing on FPGAs for multi-tenant networks, suggesting that SENS-style PDU classification could be pushed into such pipelines. Beyond the network stack, reducing per-element management overhead is a recurring theme in data-intensive systems. Gan et al. [41] address this at the data-structure level with a degree-equalized sparse format for large-scale graph processing. Whether similar structure-aware optimizations can be applied across storage and graph workloads remains an open question.

7. Conclusions

This paper presents SENS, a semantic-aware scheduling layer for NVMe over TCP that addresses the performance imbalance between high-speed networks and host-side software stacks. To mitigate interrupt overhead and protocol serialization observed in the native NVMe over TCP implementation under high concurrency, SENS introduces an asymmetric dual-threshold batching mechanism.

SENS aggregates discrete PDUs into memory vectors to reduce interrupt overhead, while exposing NVMe-oF PDU semantics to the batching layer so that time-sensitive control flows bypass the accumulation window. This asymmetric dispatch reduces control-message blocking caused by bulk aggregation.

Evaluation of the Linux kernel prototype on commodity servers shows improved compute efficiency compared to the unmodified native driver. SENS reaches the physical throughput limit of the tested NVMe SSD using 4–5 CPU cores, whereas the native NVMe over TCP implementation requires approximately twice as many cores under the same workload. RAMDisk-based stress tests show up to a $2.5\times$ increase in peak I/O throughput, while end-to-end latency remains bounded under both device-limited and CPU-limited conditions. SENS shows that incorporating storage-level semantics into PDU batching resolves scalability bottlenecks, delivering resource efficiency at no additional hardware cost.

Author Contributions: Conceptualization, X.Q. and X.X.; formal analysis, X.Q.; investigation, L.T.; methodology, X.Q.; resources, L.T. and G.H.; software, X.Q.; supervision, X.X.; validation, X.Q., L.T. and G.H.; writing—original draft preparation, X.Q.; writing—review and editing, X.X. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Natural Science Foundation of China under grant number 62402504.

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

NVMe	Non-Volatile Memory Express
NVMe-oF	NVMe over Fabrics
PDU	Protocol Data Unit
R2T	Ready to Transfer
H2CData	Host to Controller Data
C2HData	Controller to Host Data
PG	PDU Group
IOPS	Input/Output Operations Per Second
DPU	Data Processing Unit
NIC	Network Interface Card
RDMA	Remote Direct Memory Access
RoCEv2	RDMA over Converged Ethernet version 2
PFC	Priority Flow Control
SPDK	Storage Performance Development Kit
SSD	Solid State Drive
HPC	High-Performance Computing

References

1. Klimovic, A.; Litz, H.; Kozyrakis, C. ReFlex: Remote flash $\approx$ local flash. In Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Xi'an, China, 4–8 April 2017; pp. 345–359.
2. Vuppapapati, M.; Miron, J.; Agarwal, R.; Truong, D.; Motivala, A.; Cruanes, T. Building an elastic query engine on disaggregated storage. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20), Santa Clara, CA, USA, 25–27 February 2020; pp. 445–458.
3. Sivakumar, S. Performance Characterization Study of NVMe Storage Over TCP. Master's Thesis, Vrije Universiteit Amsterdam and Universiteit van Amsterdam, Amsterdam, The Netherlands, 2024.
4. Guz, Z.; Li, H.; Shayesteh, A.; Balakrishnan, V. Performance characterization of NVMe-over-fabrics storage disaggregation. *ACM Trans. Storage* **2018**, *14*, 31. [[CrossRef](#)]
5. Boukhobza, J.; Olivier, P.; Lim, W.S.; Chen, L.-C.; Hsieh, Y.-S.; Wu, S.-T.; Ho, C.-C.; Huang, P.-C.; Chang, Y.-H. A survey on flash-memory storage systems: A host-side perspective. *ACM Trans. Storage* **2025**, *21*, 24. [[CrossRef](#)]
6. Kang, Y.; Liu, M. Understanding and Profiling NVMe-over-TCP Using ntprof. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*, Philadelphia, PA, USA, 28–30 April 2025; USENIX Association: Berkeley, CA, USA, 2025; pp. 1117–1136.
7. Sun, X.; Zhang, M.; Shan, Y.; Chen, K.; Jiang, J.; Wu, Y. Scalio: Scaling up DPU-based JBOF key-value store with NVMe-oF target offload. In Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25), Boston, MA, USA, 7–9 July 2025.
8. Xu, J.; Qiu, Y.; Chen, Y.; Wang, Y.; Lin, W.; Lin, Y.; Zhao, S.; Liu, Y.; Wang, Y.; Chen, W. Performance characterization of SmartNIC NVMe-over-fabrics target offloading. In Proceedings of the 17th ACM International Systems and Storage Conference (SYSTOR), Virtual, Israel, 16 September 2024; pp. 14–24.

9. Zhu, Y.; Dhakal, A.; Bruel, P.; Rattihalli, G.; Xiao, Y.; Lombardi, J.; Milojicic, D. An RDMA-first object storage system with SmartNIC offload. In Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, St. Louis, MO, USA, 16 November 2025; pp. 1650–1657.
10. Jia, Y.; Anger, E.; Chen, F. When NVMe over fabrics meets Arm: Performance and implications. In Proceedings of the 35th Symposium on Mass Storage Systems and Technologies (MSST), Santa Clara, CA, USA, 20–24 May 2019; pp. 134–140.
11. Hwang, J.; Cai, Q.; Tang, A.; Agarwal, R. TCP $\approx$ RDMA: CPU-efficient remote storage access with i10. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20), Santa Clara, CA, USA, 25–27 February 2020; pp. 127–140.
12. Hwang, J.Y.; Park, C.; Yeom, I.; Kim, Y. NetCAS: Dynamic cache and backend device management in networked environments. *arXiv* **2025**, arXiv:2510.02323.
13. Lightbits Labs. NVMe/TCP Drivers and Kernel Support. Available online: <https://www.lightbitlabs.com/nvme-tcp-drivers/> (accessed on 5 March 2025).
14. Guo, C.; Wu, H.; Deng, Z.; Soni, G.; Ye, J.; Padhye, J.; Lipshteyn, M. RDMA over commodity ethernet at scale. In Proceedings of the 2016 ACM SIGCOMM Conference, Florianopolis, Brazil, 22 August 2016; pp. 202–215.
15. Mittal, R.; Shpiner, A.; Panda, A.; Zahavi, E.; Krishnamurthy, A.; Ratnasamy, S.; Shenker, S. Revisiting network support for RDMA. In Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, Budapest, Hungary, 7 August 2018; pp. 313–326.
16. Yu, J.; Liu, G.; Li, X.; Dong, W.; Li, Q. Cross-layer Coordination in the I/O Software Stack of Extreme-scale Systems. *Concurr. Comput. Pract. Exp.* **2018**, *30*, e4396. [[CrossRef](#)]
17. Koh, S.; Lee, C.; Kwon, M.; Jung, M. Exploring system challenges of ultra-low latency solid state drives. In Proceedings of the 10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 18), Boston, MA, USA, 9–10 July 2018.
18. Zhang, J.; Kwon, M.; Gouk, D.; Koh, S.; Lee, C.; Alian, M.; Chun, M.; Kandemir, M.T.; Kim, J.; Jung, M. FlashShare: Punching through server storage stack from kernel to firmware for ultra-low latency SSDs. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18), Carlsbad, CA, USA, 8–10 October 2018; pp. 477–492.
19. Xie, X.; Xiao, L.; Wei, D.; Li, Q.; Song, Z.; Ge, X. Pinpointing and scheduling access conflicts to improve internal resource utilization in solid-state drives. *Front. Comput. Sci.* **2019**, *13*, 35–50. [[CrossRef](#)]
20. Kourtis, K.; Ioannou, N.; Koltsidas, I. Reaping the performance of fast NVM storage with uDepot. In Proceedings of the 17th USENIX Conference on File and Storage Technologies (FAST '19), Boston, MA, USA, 25–28 February 2019; pp. 1–15.
21. Peng, J.; Yang, L.; Wu, H.; Zhang, W.; Wu, Z.; Zhang, W.; Li, J.; Dai, Y.; Dong, Y. A Survey on Machine Learning-Based HPC I/O Analysis and Optimization. *IEEE Trans. Parallel Distrib. Syst.* **2026**, *37*, 618–632. [[CrossRef](#)]
22. Hwang, J.; Vuppapapati, M.; Peter, S.; Agarwal, R. Rearchitecting Linux storage stack for μ s latency and high throughput. In Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI '21), Santa Clara, CA, USA, 14–16 July 2021; pp. 113–128.
23. Jia, W.; Jiang, D.; Xiong, J. CetoFS: A high-performance file system with host-server collaboration for remote storage. In Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST '26), Santa Clara, CA, USA, 24–26 February 2026.
24. Bjørling, M.; Axboe, J.; Nellans, D.; Bonnet, P. Linux block IO: Introducing multi-queue SSD access on multi-core systems. In Proceedings of the 6th International Systems and Storage Conference (SYSTOR '13), Haifa, Israel, 30 June–2 July 2013; pp. 1–14.
25. Lee, G.; Shin, S.; Song, W.; Ham, T.J.; Lee, J.W.; Jeong, J. Asynchronous I/O stack: A low-latency kernel I/O stack for ultra-low latency SSDs. In Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC 19), Renton, WA, USA, 10–12 July 2019; pp. 603–616.
26. Yu, Y.J.; Shin, D.I.; Shin, W.; Song, N.Y.; Choi, J.W.; Kim, H.S.; Eom, H.; Yeom, H.Y. Optimizing the block I/O subsystem for fast storage devices. *ACM Trans. Comput. Syst.* **2014**, *32*, 1–48. [[CrossRef](#)]
27. NVM Express. NVM Express, Revision 1.4. Available online: https://nvmexpress.org/wp-content/uploads/NVM-Express-1_4-2019.06.10-Ratified.pdf (accessed on 1 July 2025).
28. Kim, J.; Jang, I.; Reda, W.; Im, J.; Canini, M.; Kostić, D.; Kwon, Y.; Peter, S.; Witchel, E. LineFS: Efficient SmartNIC offload of a distributed file system with pipeline parallelism. In Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP '21), Virtual Event, 26–29 October 2021; pp. 756–771.
29. Zhu, W.; Shen, Z.; Wei, Q.; Chen, R.; Yao, X.; Yu, D.; Shao, Z. HiDPU: A DPU-oriented hybrid indexing scheme for disaggregated storage systems. In Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST '25), Santa Clara, CA, USA, 25–27 February 2025; pp. 271–285.
30. Shu, J.; Qian, K.; Zhai, E.; Liu, X.; Jin, X. Burststable cloud block storage with data processing units. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), Santa Clara, CA, USA, 10–12 July 2024; pp. 783–799.

31. Wei, X.; Cheng, R.; Yang, Y.; Chen, R.; Chen, H. Characterizing off-path SmartNIC for accelerating distributed systems. In Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23), Boston, MA, USA, 10–12 July 2023; pp. 987–1004.
32. Cheng, X.; Chen, Y.; Yang, X.; Zhou, H.; Luo, L.; Guo, D. Memory-Efficient Programmable Packet Parsing for Multi-Tenant Terabit Networks. *Comput. Netw.* **2025**, *264*, 111240. [[CrossRef](#)]
33. Ousterhout, A.; Fried, J.; Behrens, J.; Belay, A.; Balakrishnan, H. Shenango: Achieving high CPU efficiency for latency-sensitive datacenter workloads. In Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19), Boston, MA, USA, 26–28 February 2019; pp. 361–378.
34. Kadekodi, R.; Lee, S.K.; Kashyap, S.; Kim, T.; Kolli, A.; Chidambaram, V. SplitFS: Reducing software overhead in file systems for persistent memory. In Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP '19), Huntsville, ON, Canada, 27–30 October 2019; pp. 494–508.
35. Kaufmann, A.; Stamler, T.; Peter, S.; Sharma, N.K.; Krishnamurthy, A.; Anderson, T. TAS: TCP acceleration as an OS service. In Proceedings of the 14th EuroSys Conference (EuroSys '19), Dresden, Germany, 25–28 March 2019; pp. 1–16.
36. Salim, J.H.; Olsson, R.; Kuznetsov, A. Beyond Softnet. In Proceedings of the 5th Annual Linux Showcase & Conference, Oakland, CA, USA, 5–10 November 2001; pp. 165–172.
37. Salah, K.; El-Badawi, K.; Haidari, F. Performance Analysis and Comparison of Interrupt-Handling Schemes in Gigabit Networks. *Comput. Commun.* **2007**, *30*, 3425–3441. [[CrossRef](#)]
38. Han, J.-I.; Kim, Y.-M.; Lee, J. Adaptive Interrupt Coalescing for Energy Efficiency and Performance. In *Proceedings of the 2015 International Conference on Information and Communication Technology Convergence (ICTC)*; IEEE: Jeju Island, Republic of Korea, 2015; pp. 41–46.
39. Yu, J.; Liu, G.; Dong, W.; Li, X. WatCache: A Workload-Aware Temporary Cache on the Compute Side of HPC Systems. *J. Supercomput.* **2019**, *75*, 554–586. [[CrossRef](#)]
40. Tai, A.; Smolyar, I.; Wei, M.; Tsafirir, D. Optimizing storage performance with calibrated interrupts. *ACM Trans. Storage* **2022**, *18*, 1–32. [[CrossRef](#)]
41. Gan, X.; Li, T.; Gong, C.; Li, D.; Dong, D.; Liu, J.; Lu, K. GraphCSR: A Degree-Equalized CSR Format for Large-Scale Graph Processing. *Proc. VLDB Endow.* **2025**, *18*, 4255–4268. [[CrossRef](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.